

Project: RNA Sequence Generator via Reinforcement Learning with Memory-Augmented Architectures

Objective

Design a reinforcement learning agent capable of generating biologically meaningful RNA sequences under various structural and compositional constraints. The system was built to:

- Maximize **GC content balance**
- Target **specific motifs** at randomized positions
- Promote **sequence diversity**
- Avoid **repetition**
- Integrate **RNA secondary structure prediction** using *ViennaRNA* as a non-differentiable oracle

We also benchmarked the impact of different **RNN memory mechanisms** (LSTM, GRU, CARU, and no-memory) on the agent's ability to learn these sequence objectives.

Initial Setup

We started with a basic differentiable RL simulator where:

- An agent generated sequences one base at a time.
- Rewards were soft and focused on GC balance.
- Inputs included the last predicted base + an optional memory state.
- The action space was a softmax over 4 bases (A, C, G, U).

At this stage, we trained a simple model using just **GC ratio reward** with fixed targets.

Major Extensions and Changes

1. Reframed the Environment as Pure Reinforcement Learning

Originally the state included the full generated sequence, which led to inconsistent tensor shapes. We simplified this:

- Each state only includes the last predicted base + memory.
- This ensures fixed-size input vectors and pure state-to-action transitions.

Why:

Tensor shape mismatch errors and the need for fully differentiable environment-step transitions made the previous method infeasible. The RL formulation better supports batch processing and memory models.

2. Memory-Augmented Policy Architectures

We tested different memory architectures for partial observability and long-term planning:

- **LSTM** (Full_LSTM): Maintains hidden + cell state.
- **GRU Variants**: Full_GRU and Minimal_GRU.
- **CARU**: Controlled Additive Recurrent Unit for gated memory update.
- **No Memory**: Baseline MLP policy.

Each architecture took as input the one-hot base + memory and output logits for the next base + memory update.

3. Combined Multi-Objective Reward Function

We introduced a **modular reward signal** combining:

- **GC Content Reward**: Target randomly varied per batch between 0.4–0.6.
- **Motif Reward**: Target nucleotide (e.g., A) at random position (e.g., pos 12).
- **Structural Entropy Reward**: Maximize entropy of base probabilities as a folding complexity proxy.
- **Repetition Penalty**: Penalize if the same base repeats (e.g., CCCCC).
- **Oracle Reward** (*ViennaRNA*): Negative free energy reward computed at the end of the sequence.
- **Entropy Bonus**: Encourage diverse predictions.
- **KL Regularization**: Prevent collapse to uniform or repetitive sequences.

Why:

Biological sequences are not defined by one factor. These additions make the task more biologically grounded and computationally non-trivial.

4. Oracle Integration via ViennaRNA

At the final timestep, the full generated sequence is passed to `RNAfold` via subprocess:

python

CopyEdit

```
subprocess.run(["RNAfold", temp_path])
```

We parse the output to extract folding free energy and assign a final reward:

python

CopyEdit

```
oracle_reward = -1 * energy # Lower energy = more stable
```

Why:

This introduces **sparse**, **non-differentiable**, and **delayed rewards** — perfect for evaluating real RNA

structural stability.

5. Visualization & Analysis Tools

- **Training Plots:** Reward and loss over time.
 - **GC Content Distribution:** Histogram of final GC content.
 - **Heatmaps:** Nucleotide frequency per position (A, C, G, U).
 - **Sequence Sampling:** Decoded sampled sequences shown per batch.
 - **Oracle Logging:** Folding energies from ViennaRNA were optionally saved.
-

Challenges and Fixes

- **Reward = -loss** pattern led to confusing logs. We clarified that `loss = -reward` is the policy gradient formulation.
 - **Invalid sequences** or empty outputs were fixed by better `argmax` sampling and decoding logic.
 - **Shape mismatches** were resolved by ensuring fixed-size states and strict timestep control in `expand_full_trajectory`.
 - **Oracle reward errors** (due to subprocess or temp file access) were fixed with safe exception handling and temp file cleanup.
-

Final Pipeline Structure

1. **Randomized Batch Targets:** New GC targets and motif positions every episode.
 2. **Trajectory Sampling:** Each agent generates a sequence one step at a time.
 3. **Memory Manipulation:** Action outputs include memory-update vectors for policy-specific memory types.
 4. **Reward Aggregation:** At every timestep, reward terms are combined.
 5. **Final Oracle Call:** At the end, ViennaRNA computes free energy.
 6. **Training:** Policy is updated via REINFORCE with entropy and KL regularization.
-

Experimental Results

- **LSTM and GRU-based agents** consistently outperformed no-memory policies.
 - **CARU** offered strong diversity and structure generation with fewer parameters.
 - **Oracle rewards** helped in generating more stable RNA structures.
 - **Motif targeting** was successful even under noisy objectives.
 - **Repetition was reduced** through entropy regularization and penalty terms.
-

Next Steps (Future Work)

- Integrate **RNAstructure** or **EternaFold** as more advanced oracles.
- Use **PPO** or **Actor-Critic** for more stable policy updates.
- Allow **inverse folding tasks**: generate RNA that folds into a given structure.
- Add **curriculum learning** to progressively scale motif complexity and target constraints.